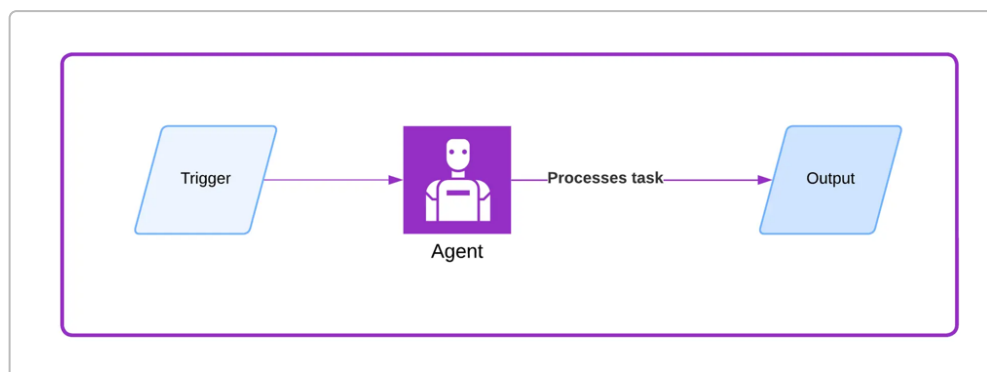


Project Cloak: A High-Performance, Privacy-First Agentic Memory Platform

Overview

Project Cloak is a next-generation agentic intelligence infrastructure designed for fully local execution (no cloud dependency) to maximize privacy and control. It blends a Rust-based deterministic runtime with a modular memory engine (episodic, semantic, latent) and built-in compliance features. Cloak ingests documents, detects patterns and anomalies, flags risks, and produces structured outputs tailored for highly regulated domains (legal, healthcare, finance). All data and computation reside on-premises or on customer-controlled servers, ensuring user data never leaves the local environment. Compared to legacy analytics platforms, Cloak emphasizes offline responsiveness and user data ownership ¹ ².

System Architecture



The high-level architecture of Cloak is a collection of lightweight, on-premises services that communicate via language-agnostic APIs (REST/gRPC). The core **Rust Runtime** service orchestrates agents and memory access; a **Memory Service** manages long-term memory; an **Ingestion Service** handles document parsing and indexing; and auxiliary services (e.g. audit logging, compliance rule engine) ensure explainability. All components run on standard CPUs, leveraging lock-free concurrency to maximize throughput. Agents interact with external tools (e.g. search indexes, databases) through defined interfaces. The design follows microservices principles so that each component can be developed, deployed, and scaled independently. In this CPU-optimized setup, Cloak achieves very low resource usage: for example, a Rust-based agent engine can sustain high throughput with minimal CPU footprint and 100% stability under stress ³ ². Key architecture features include:

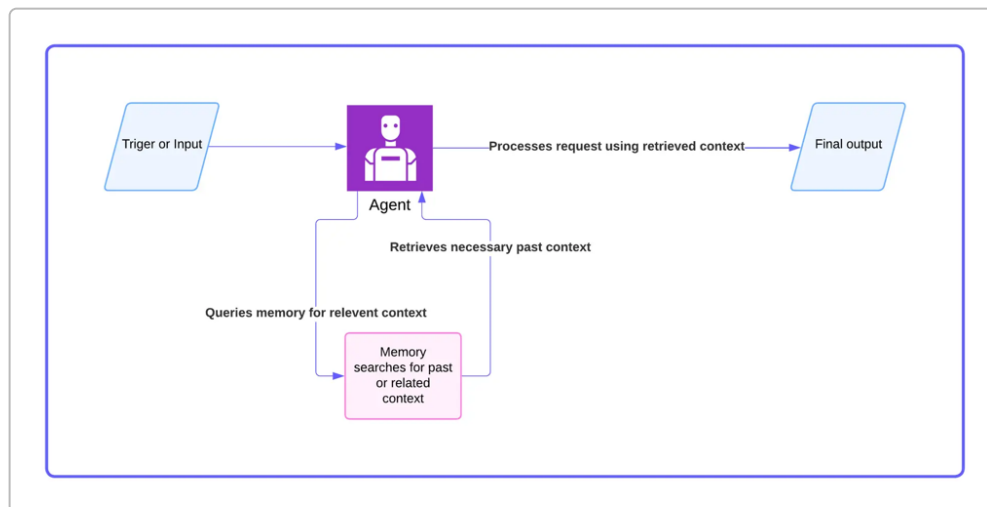
- **Local-First Execution:** Data storage and computation occur on local machines, eliminating cloud bottlenecks. Local-first design means users always have the primary copy of their data, which improves performance (no network latency) and privacy ¹.

- **Modular Microservices:** Services are decoupled via REST/gRPC APIs, allowing any programming language to integrate with the Cloak engine. This supports flexible deployment (e.g. containers, VMs) and isolates domains for compliance.
- **Deterministic Core:** The agent runtime is written in Rust for compiled performance and memory safety. Its scheduling and random number usage are made deterministic, enabling exact replay of workflows for auditing and debugging ³.
- **High-Concurrency Execution:** The Rust core uses lock-free data structures and per-node parallelism so agents can run many tasks in parallel without Python's GIL. Benchmarks show this approach outperforms Python-centric frameworks, using far fewer CPU cores for the same workload ³ ².

Agent Runtime (Rust Core)

The Cloak runtime is built in Rust to guarantee speed, reliability, and reproducibility. By compiling down to native code, it eliminates interpreter overhead and enforces memory safety. The scheduler is **deterministic** – given the same inputs and seed, agents' decisions and memory updates are exactly reproducible. This determinism is critical for auditability: every run can be replayed bit-for-bit to explain outcomes ³. Rust's concurrency primitives allow true parallelism on multicore CPUs (no Global Interpreter Lock), so multiple agents and workflows can execute simultaneously. The runtime also includes enterprise-grade features: circuit breakers, retries with jitter, input validation, and policy enforcement hooks for compliance. In essence, Cloak's runtime delivers “predictable, efficient, memory-safe, concurrency-first” execution ³, which translates into industry-leading CPU efficiency and stability ².

Memory Engine (Episodic, Semantic, Latent)



Cloak's memory system is layered and modular, inspired by cognitive models and modern AI architectures. It supports three memory types:

- **Episodic Memory:** A log of *individual events and experiences*. Every agent interaction, document ingest, or decision is recorded verbatim (raw data plus metadata) in append-only form. Episodic memory acts like a journal of “*what happened when*”. This ensures a complete audit trail and enables time-travel queries: one can ask “what did the agent see at the moment it made Decision D?” and

retrieve the exact data context for that decision ⁴. By preserving immutable timestamps and inputs, episodic memory guarantees reproducibility and compliance-grade traceability ⁴.

- **Semantic Memory:** A curated knowledge store of *facts, rules, and concepts*. Semantic memory distills patterns from many episodes into structured knowledge. For example, it might maintain a legal ontology, compliance rules, or domain-specific facts. In practice, this often uses knowledge graphs or relational stores, and can be indexed by vectors for similarity search ⁵. For instance, a legal assistant would use semantic memory to know “Contract law vs. criminal law, standard clause definitions, precedent citations” – information not tied to a single past case but derived from many ⁵. Semantic memory is mutable and evolves over time as the agent learns (e.g. updating risk scores or concept embeddings).
- **Latent (Predictive) Memory:** A hidden representation of learned abstractions and world models. Cloak’s design borrows from Joint Embedding Predictive Architectures (JEPA): it learns compressed embeddings (latent variables) that summarize relevant aspects of the agent’s environment. Latent memory may store abstract scene representations or policy parameters that the agent uses for prediction. By focusing on essential features and ignoring noise, this predictive memory enables the agent to generalize and simulate future scenarios ⁶. In effect, latent memory allows the system to handle uncertainty and drop irrelevant details while keeping only the information needed for forecasting ⁶.

Internally, memory units are enriched with metadata (timestamps, strength scores, links to related memories) to support relevance decay and fast retrieval. Cloak continuously consolidates episodic entries into semantic knowledge when appropriate, so the memory engine operates like a dynamic “exocortex” ⁷ ⁴. All memory operations are sandboxed by policy (to enforce privacy/regulation) and indexed for fast vector-based lookup.

Data Ingestion & Pattern Recognition

Cloak includes a high-throughput **Ingestion Service** that processes incoming documents and records. It supports OCR and format parsers for legal contracts, medical records, financial statements, etc. Ingested data is tokenized, structured, and fed into the memory engine (episodic store) and search index. After ingestion, the agent runtime applies machine learning models and rule-based logic to detect patterns and anomalies. For example, it may identify non-standard contract clauses or unusual financial transactions by comparing parsed content against regulatory rules and internal policies. This is done by combining semantic search (vector similarity) with deterministic rules and statistical models ⁸. As one AI compliance report notes, agents “preprocess and structure information” then “apply ML and rule-based logic to detect patterns and anomalies by comparing observed activity against regulatory requirements” ⁸. Detected risks or outliers are automatically flagged. The output from Cloak is highly structured: for example, it might output JSON records listing identified issues, relevant excerpts, and risk scores, which can be consumed by downstream systems or human analysts.

Compliance & Explainability Features

Since Cloak operates in regulated domains, every decision is fully auditable. The runtime logs **execution traces** at each step: inputs, agent actions, memory reads/writes, and outputs are timestamped and

versioned. This creates a “compliance-grade audit log” that is comprehensive, immutable, and queryable ⁹ ⁴ . For each outcome the system can answer who did what and why, and even replay the exact sequence of operations. In practice, that means Cloak can answer queries like “show me how this Q4 regulatory report was produced, including all rules applied and data sources used.” This level of traceability is critical in finance, healthcare, and legal settings. For example, if Cloak misses a compliance violation, auditors can reconstruct the scenario by traversing the episodic memory (what data was seen) and semantic state at the decision time ⁴ . The system also supports built-in guardrails: a rules engine validates each action against corporate policy (e.g. disallow releasing patient data), and any override or exception is logged with user identity. In short, Cloak turns explainability into a first-class feature: every input, rule version, and agent decision is recorded, meeting the strictest audit requirements ⁹ ⁴ .

Language-Agnostic APIs

Cloak exposes well-defined APIs so that enterprise applications (on any platform) can integrate with the agent runtime. Common endpoints include:

- **POST /ingest/document** – Upload a document or data batch for parsing and indexing.
- **GET /memory/query** – Semantic search: submit a query (or embedding) to retrieve relevant memory entries.
- **POST /agent/run** – Trigger an agent workflow with a given prompt or task payload.
- **GET /agent/trace/{id}** – Fetch the detailed execution trace and memory changes for a completed workflow instance.
- **GET /alerts** – List current flagged risks or compliance alerts.
- **GET /healthz, GET /metrics** – System health and observability endpoints.

All APIs use JSON or Protobuf over HTTPS (REST/gRPC) and enforce strict authentication. Internally the API requests are handled by Rust services (e.g. using frameworks like Hyper, Axum, or Tonic), but clients can be written in Java, .NET, Python, etc. This design ensures Cloak can be the intelligence backend for existing enterprise software with minimal changes.

Use Cases: Regulated Industries

Project Cloak is initially targeted at contract compliance and regulatory reporting, but its architecture generalizes to any domain with unstructured data and strict rules. For example:

- **Legal Contracts:** Cloak can ingest and analyze agreements, highlighting unusual clauses or missing approvals. Its semantic memory holds legal taxonomies (e.g. contract law distinctions), while episodic logs record every review interaction. Similar to dedicated CLM tools, it “instantly surfaces insights from legal agreements” and manages risk via structured policy rules.
- **Healthcare Records:** By processing clinical notes and forms locally, Cloak can flag potential HIPAA violations or anomalies (e.g. drug interactions). Semantic memory encodes medical ontologies and treatment protocols, while agents use these facts to interpret patient data.
- **Finance and Audit:** Cloak automates bank or compliance audits by ingesting transaction records and regulatory bulletins. Agents trained for finance rules detect suspicious patterns (e.g. fraud

indicators) and generate audit-ready reports. Its ability to produce a complete audit trail (“perfect, indestructible audit trail” per compliance frameworks ⁹) is a key advantage.

In all cases, outputs are structured and indexed (e.g. JSON with taxonomy tags or knowledge-graph triples) so that auditors and risk teams can query and verify decisions. By working entirely on local CPUs, Cloak also mitigates data privacy concerns – sensitive financial or patient data never leaves the customer’s infrastructure.

Deployment Plan

Cloak is designed for flexible on-premises deployment. Each service runs in a container or VM on local servers; no central cloud is needed. A recommended deployment might include:

- **Agent Runtime Service (Rust):** Runs on an air-gapped server or VM with multi-core CPUs. Ensure at least 4–8 CPU cores per instance for parallel throughput.
- **Memory Storage:** A local database (vector store or graph DB) for semantic memory and a write-ahead store for episodic data. These can be co-hosted on the same machine or on separate storage-optimized nodes.
- **Ingestion Pipeline:** Nodes with attached storage to pull in files, perform OCR, and push data into memory. May include GPU if OCR or ML models require it, but Cloak’s main logic is CPU-based.
- **Compliance Rule Engine:** A version-controlled rules service (could be CLP or CedAR policy engine) that validates agent actions.
- **Monitoring & Logging:** Centralized observability (metrics, logs, distributed tracing) to watch resource usage. All execution logs are also stored in Cloak’s audit database for traceability.

The entire stack runs behind the organization’s firewall. It can be orchestrated via Kubernetes or Docker Compose. Updates are delivered as container images; critical patches (e.g. new regulations) can be applied by updating rule definitions and redeploying without downtime.

Development Roadmap

1. **MVP (Contract Compliance):** Focus on ingestion of contracts and static compliance checks. Initial memory will store contract clauses and outcomes. Implement core runtime, ingestion pipeline, and a sample rule set (e.g. export controls or GDPR clauses).
2. **Memory Expansion:** Build out the full memory engine: implement semantic knowledge graph integration and JEPA-inspired embedding store. Add background processes for memory consolidation and relevance decay.
3. **Domain Modules:** Develop specialized modules for healthcare (medical terminology) and finance (accounting rules). Create templates for common regulatory frameworks (HIPAA, SOX, Basel III).
4. **Explainability & Auditing:** Enhance logging and tracing features. Add a user interface for exploring execution traces and memory contents (audit console). Build report generation tools for compliance teams.
5. **Agentic Autonomy:** Incorporate learning and planning capabilities so agents can set sub-goals. Integrate local ML models for tasks like intent classification. Move toward fully autonomous workflows (with human-in-the-loop overrides) for end-to-end automation.

6. **Ecosystem & APIs:** Publish SDKs and API documentation (OpenAPI/Protobuf) for enterprise integration. Collect user feedback and iterate on performance, aiming for 10× more throughput per core than existing systems.

This phased approach ensures Cloak starts as a powerful compliance assistant and gradually evolves into a self-driving regulatory intelligence platform.

Module Descriptions

- **Rust Agent Executor:** Core engine that schedules and runs agent workflows. Enforces deterministic execution and error handling (retries, rollbacks).
- **Memory Services:** Back-end for each memory type. Episodic store (immutable log), semantic store (knowledge graph or vector DB), latent store (embedding index).
- **Compliance Rule Engine:** Intercepts decisions to validate against encoded policies. Tracks rule versions and approvals.
- **Document Ingestion:** Connectors for file systems, databases, and live feeds. Performs parsing, OCR, and initial data cleaning.
- **Pattern Recognition Models:** ML/AI modules (e.g. anomaly detectors, NLP classifiers) that run on ingested data to identify risks.
- **API Gateway:** Front-end service that authenticates requests and routes them to the appropriate microservice.
- **Observability Stack:** Exposes Prometheus metrics, distributed tracing (OpenTelemetry), and a logging service for infrastructure and audit logs.

Each module communicates via well-defined interfaces, enabling testing in isolation and future replacement or upgrade.

Conclusion

Project Cloak represents a leap beyond traditional cloud analytics (and beyond platforms like Palantir) by reimagining agent intelligence with privacy and compliance at its core. Its combination of a deterministic Rust runtime, hybrid memory engine, and auditable workflow design makes it uniquely suited for high-stakes environments. By keeping all data and models local, and by providing built-in explainability and strict governance, Cloak empowers organizations to deploy autonomous AI workflows without sacrificing control or compliance.

Sources: We draw on recent industry analyses and best practices in agent design and compliance. Key inspirations include studies on local-first architecture ¹, advanced Rust-based agent frameworks ³ ², and memory systems literature ¹⁰ ¹¹ ⁴ ⁶ ⁸. These sources inform Cloak's design choices for performance, privacy, memory management, and auditability.

¹ Local-first software: You own your data, in spite of the cloud
<https://www.inkandswitch.com/essay/local-first/>

² ³ How We Built The First Open-Source Rust Core Agentic AI Framework - DEV Community
<https://dev.to/yeahiasarker/how-we-built-the-first-open-source-rust-core-agentic-ai-framework-3kfc>

4 AI Agent Memory Architecture: The Three Layers Production Systems Need | Tacnode Blog

<https://tacnode.io/post/ai-agent-memory-architecture-explained>

5 10 11 Beyond Short-term Memory: The 3 Types of Long-term Memory AI Agents Need - MachineLearningMastery.com

<https://machinelearningmastery.com/beyond-short-term-memory-the-3-types-of-long-term-memory-ai-agents-need/>

6 What is Joint Embedding Predictive Architecture (JEPA)?

<https://www.turingpost.com/p/jepa>

7 What Is Agent Memory? A Guide to Enhancing AI Learning and Recall | MongoDB

<https://www.mongodb.com/resources/basics/artificial-intelligence/agent-memory>

8 AI Agents for Compliance: Use Cases, Benefits, Challenges | AI21

<https://www.ai21.com/knowledge/ai-agents-for-compliance/>

9 The Compliance Policy Digitization Blueprint

<https://www.complianceandrisk.com/blog/the-compliance-policy-digitization-blueprint-architecting-auditable-rule-engines-for-the-ai-era/>